OpsSkill: Autonomous Operations Skill Learning for Cloud-Native Systems

Anonymous
Under Review

***Abstract*—As large language models (LLMs) demonstrate increasing capability in tool invocation and autonomous agents, leveraging them for automated operations in DevOps and SRE scenarios has become an important research direction. However, existing methods mostly model operations tasks as free-form command generation or interactive tool-calling processes, lacking unified modeling of preconditions, action constraints, success criteria, rollback mechanisms, and risk boundaries, making it difficult to achieve safe, reliable, and reusable automated execution in real infrastructure environments.**

This paper proposes OpsSkill, an autonomous operations skill learning framework for cloud-native systems. OpsSkill automatically compiles task cards into structured skill representations containing preconditions, action sequences, hidden verifiers, rollback plans, and risk labels; during execution, it controls skill deployment through policy constraints and capability probing; during optimization, it continuously improves skills based on structured execution reports and failure signatures. Unlike evaluations that rely solely on command return values, OpsSkill further designs a multi-signal hidden verification mechanism for real remote environments, combining object state, events, logs, and service health information to determine whether a skill has truly accomplished its operational objective. The framework's core designs (Typed Skill IR, gated execution, hidden verification) are platform-agnostic and applicable to Kubernetes, OpenStack, AWS, and other cloud-native platforms.

We validate OpsSkill on a Kubernetes cluster (3 nodes, K8s v1.33.6) as a representative platform, injecting CPU stress, memory stress, and network delay faults via Chaos Mesh across 8 operations tasks. We compare against direct command generation, ReAct-style agents, Reflexion-style optimization, and template retrieval as four baselines, and quantify each component's contribution through ablation studies. OpsSkill achieves a 90% composite score, outperforming direct command generation (25%), ReAct (40%), Reflexion (40%), and template retrieval (25%) by 50–65 percentage points, while maintaining zero unsafe action rate. Ablation studies show that Typed Skill IR contributes 65% of the score improvement, hidden verification contributes 30%, and policy gating reduces the unsafe action rate from 12% to 0% at a cost of only 1% in score.

***Index Terms*—Autonomous operations, cloud-native systems, large language models, skill learning, safe execution**

I. Introduction

With the widespread deployment of cloud-native systems and microservice architectures [1], DevOps and SRE teams must continuously handle a large volume of repetitive yet high-risk operations tasks—fault diagnosis, configuration remediation, service recovery, scaling adjustments, and permission anomaly handling [2]. These tasks typically rely on operators' comprehensive understanding of system state, toolchains, failure modes, and organizational constraints, exhibiting pronounced state dependency, environment heterogeneity, and execution risk. Ghosh et al. [3] show through an empirical study of Microsoft Teams' large-scale cloud services that over 60% of high-severity incidents involve multi-step diagnosis and remediation workflows, making automation imperative. Recent advances in LLM reasoning [4], code generation, and tool calling [5] make it attractive to build autonomous agents that can assist or partially automate these tasks [6].

However, existing methods remain far from truly usable autonomous operations. On one hand, many approaches simplify operations tasks into free-form command generation or interactive tool-calling [7], ignoring that operations actions are heavily constrained and potentially costly. For real production or pre-production environments, an incorrect command sequence may not only cause task failure but also lead to cross-scope impacts, service disruptions, or even irreversible data corruption [8]. On the other hand, existing evaluations typically rely on visible command return codes or single check items to determine task completion [9], but in real systems, "command success" does not equate to "service recovery." For example, a Pod being successfully restarted does not mean the dependency chain has recovered; a Deployment rollout completion does not mean the user-facing error rate has decreased. Therefore, autonomous operations systems need not only to "call tools" but also to possess capabilities for safe execution, hidden verification, and post-failure optimization.

To address these challenges, we propose **OpsSkill**, an autonomous operations skill learning framework for cloud-native systems. OpsSkill's core idea is to elevate operations tasks from one-shot command generation to structured skill learning. The framework's core designs (Typed Skill IR, gated execution, hidden verification, failure-driven optimization) are platform-agnostic, applicable to Kubernetes, OpenStack, AWS, and other cloud-native platforms. Specifically, we compile a task into a typed skill representation with preconditions, action sequences, success criteria, rollback plans, and risk labels; at the execution layer, we design policy-constrained gated execution with capability probing; at the evaluation layer, we introduce multi-signal hidden verifiers combining object state, events, logs, and service health to determine true task completion; at the optimization layer, we use structured execution reports and failure signatures [10] to continuously improve skill quality.

Contributions:

1) A **Typed Ops Skill IR** that compiles task cards into structured skill objects with preconditions, actions, success criteria, rollback plans, and risk labels.

2) A **policy-gated execution framework** with adaptive risk weighting that quantifies and bounds blast radius before execution, with a formal safety guarantee (Proposition 1).
3) A **hidden multi-signal verification mechanism** combining visible and hidden checks to detect false positives.
4) A **structured failure-driven optimization mechanism** that extracts failure signatures and maps them to deterministic IR edits, with a monotonic convergence guarantee (Proposition 2).
5) Systematic evaluation on a real Kubernetes cluster (as a representative cloud-native platform) with baseline comparisons, ablation studies, unified evaluation metrics, and weight sensitivity analysis to verify conclusion robustness.

II. Related Work

Existing research most relevant to OpsSkill falls into four categories: AIOps and incident knowledge mining, microservice diagnosis and root cause localization, runbook automation, and agent tool-calling evaluation and safety governance.

AIOps and Incident Knowledge Mining. The first category focuses on extracting structured knowledge from real cloud service incidents and operations investigations. Cheng et al. [6] provide a comprehensive AIOps survey, organizing techniques into anomaly detection, root cause analysis (RCA), incident triage, and automated remediation. DeCaf [11] models performance issue diagnosis and triage in large-scale cloud services as a continuous pipeline, automatically identifying fault root causes from KPI regression through machine learning and pattern mining, and has been deployed in multiple Microsoft cloud services. Ghosh et al. [3] empirically analyze 152 high-severity incidents in Microsoft Teams, finding that automated testing tools can significantly reduce incident counts, but many incidents still require manual intervention across multiple system components. RCACopilot [12] combines LLMs with domain-specific incident pipelines, achieving 78.2% RCA accuracy for the top-10 most frequent teams in Microsoft's cloud services. These methods mostly remain at the knowledge extraction or root cause identification level, without compiling knowledge into executable skills with preconditions, rollback, and success criteria.

Microservice Diagnosis and Root Cause Localization. The second category focuses on fine-grained performance diagnosis and explainable fault localization. Roy et al. [7] systematically explore LLM-based agents for RCA, finding that agents can effectively collect and correlate diagnostic signals from different sources (logs, metrics, topology), but still face challenges in translating findings into safe remediation actions. D-Bot [13] proposes an LLM-based intelligent database diagnosis system that extracts and organizes root cause knowledge from documentation into tree structures, combined with diagnostic tool calling for multi-turn interactive diagnosis. These works primarily address "finding what's wrong" rather than "safely executing and verifying the fix in a real constrained environment." In contrast, OpsSkill focuses on the complete closed loop from diagnosis to safe repair execution.

Runbook Automation. The third category is most closely related to OpsSkill. AutoTSG [14] pioneers automated synthesis of troubleshooting guides (TSGs), analyzing over 50,000 internal TSGs' structure and execution patterns at Microsoft, using program synthesis to automatically generate executable diagnostic steps. Nissist [15] further combines LLM with TSG retrieval to build a TSG-based incident mitigation Copilot that automatically retrieves relevant TSGs during incidents and guides on-call engineers through diagnostic steps via LLM. LLexus [16] builds end-to-end AI agents that parse TSG code snippets and automatically orchestrate execution through an LLM-driven planner. Mao et al. [17] propose agent-based TSG automation through multi-turn dialogue and tool calling for more flexible troubleshooting workflows. These works demonstrate the viability of operationalizing maintenance experience but typically lack unified structured skill representations (like OpsSkill's Typed Skill IR) and rarely systematically examine safety gating and hidden verification in real remote cloud-native environments.

Agent Tool Calling and Safety Governance. The fourth category concerns safety governance of agents in tool calling and multi-step interactions. ReAct [5] proposes alternating reasoning and acting, demonstrating better decision-making than pure reasoning methods across multiple tasks. Reflexion [10] introduces linguistic self-reflection for self-improvement, a form of "linguistic reinforcement learning." For safety evaluation, ToolEmu [8] proposes an LLM-simulation-based tool sandbox that identifies risky LLM agent behaviors without actually executing dangerous operations, covering 144 safety scenarios across 36 tool categories. ToolSword [9] systematically reveals safety issues in LLM tool learning from three stages: tool input, execution, and output. SafeToolBench [18] constructs the first proactive safety evaluation benchmark for tool calling, covering risk types including environment damage and privilege escalation. These works provide important conceptual foundations for OpsSkill's safety design, but either focus on general tool-calling scenarios or primarily target passive detection of safety risks. OpsSkill provides *proactive* safety through embedded risk labels in Typed Skill IR and policy gating mechanisms, achieving proactive safety protection within the operations execution pipeline.

III. Problem Definition

This section formalizes the autonomous operations skill learning problem. We first define the operational environment and task card, then introduce the environment capability profile to characterize remote cluster constraints, present the structured skill representation and its legality constraints, and finally establish the end-to-end compile–execute–optimize loop and derive the optimization objective.

A. Environment and Task Card

We model a remote operations environment as a triple $E = (\mathcal{S}, \mathcal{O}, \mathcal{U})$, where:

- $\mathcal{S}$ is the **state space**, containing the current state and configuration of all managed objects. In Kubernetes, this

includes Pods, Deployments, Services, ConfigMaps, etc.; in other platforms, it corresponds to instances, service groups, load balancers, etc.;

- $\mathcal{O}$ is the **observable signal space**, including platform CLI outputs (e.g., `kubectl`, `openstack` CLI, `aws` CLI), container/instance logs, platform events, resource metrics, etc.;
- $\mathcal{U}$ is the **tool action space**, covering all commands executable via remote interfaces (e.g., `kubectl apply`, `helm upgrade` in Kubernetes, or `aws ecs update-service` in AWS).

The rationale for triple modeling is that it cleanly separates three core elements of operational decision-making: the system's true current state ($\mathcal{S}$), the information observable by the agent ($\mathcal{O} \subseteq \mathcal{S}$, typically a subset due to permission restrictions), and the actions available to the agent ($\mathcal{U}$). This separation enables explicit modeling of "incomplete information" and "constrained actions"—two fundamental challenges in real operations.

An operations task is defined as a **task card** $x = (d, c, g, \Omega, \mathcal{T})$, where d is a natural-language description (e.g., "diagnose and fix CPU overload"), c is fault context (e.g., alert logs, anomalous metrics), g is the desired goal state (e.g., "service instance recovers and CPU utilization drops below threshold"), Ω contains constraints (e.g., "only operate on resources within the specified scope"), and $\mathcal{T} \subseteq \mathcal{U}$ is the available tool subset. The task card design is inspired by SRE runbook practice [2], augmented with formalized constraint specifications to support automated safety checking.

B. Environment Capability Profile

In real cloud-native environments, capability differences across platforms and clusters can be enormous: some may restrict access permissions (e.g., Kubernetes RBAC or AWS IAM), some lack specific extension components, and network policies may block certain diagnostic commands. To adapt skills to heterogeneous environments, we introduce the **environment capability profile**:

$$\kappa(E) = (\kappa_{rbac}, \kappa_{resource}, \kappa_{crd}, \kappa_{network}, \kappa_{obs}) \tag{1}$$

The components are:

- κ_{rbac}: Permission set of the current execution identity (e.g., Kubernetes ServiceAccount RBAC permissions, AWS IAM policies), determining which commands can be authorized;
- $\kappa_{resource}$: Inventory of existing managed resource objects (e.g., Deployments, Services in Kubernetes; instances, security groups in cloud platforms), for verifying referenced targets exist;
- κ_{crd}: Installed platform extensions (e.g., Kubernetes CRDs, cloud-managed services), determining available advanced capabilities;
- $\kappa_{network}$: Network connectivity status, including DNS resolution, service port reachability, egress policies;
- κ_{obs}: Observability infrastructure availability, such as log collection, metric query APIs, event subscriptions.

The profile is automatically probed via lightweight commands before skill execution ($\sim$3 s). This design ensures the compiler f_θ generates environment-adapted skills rather than assuming an idealized execution environment.

C. Typed Skill IR

The core data structure is the **Typed Ops Skill IR**:

$$S = (P, A, V, R, \rho, \eta) \tag{2}$$

The design intent for each field:

- $P = \{p_1, \ldots, p_n\}$: **Precondition set**. Each p_i is an executable Boolean check (e.g., "target Deployment exists," "current user has `patch` permission"). Skills execute only when all preconditions are satisfied. Preconditions "shift left" execution failures into compile-time blocks, avoiding irreversible operations in incompatible environments.
- $A = [a_1, \ldots, a_m]$: **Ordered action sequence**. Each a_i is a concrete command (e.g., `kubectl rollout restart` or `aws ecs update-service`), executed sequentially. An ordered list is chosen over a DAG because operations actions typically have strict temporal dependencies.
- $V = (V_{vis}, V_{hid})$: **Verifier pair**. V_{vis} contains visible verification checks defined by the skill itself (e.g., "exit code is 0"), while V_{hid} contains hidden checks injected by the framework (e.g., "Pod actually enters Running state," "Service endpoint returns 200"). Separating visible and hidden verification aims to detect false positives where "command succeeds but task is actually incomplete."
- $R = [r_1, \ldots, r_k]$: **Rollback plan**. When actions fail or verification does not pass, rollback operations execute in reverse order to restore environment state—a critical safety guarantee.
- $\rho \in \{\text{low}, \text{medium}, \text{high}\}$: **Risk label**. Automatically assigned by the compiler based on action types—read-only operations are labeled low, configuration modifications medium, and delete/recreate operations high.
- η: **Instance parameters**. Runtime-bound parameters (e.g., namespace names, target object names, timeout thresholds) that enable the same skill template to serve different targets and platforms.

The **legality constraint** function ensures structural completeness:

$$\Gamma(S) = \mathbb{I}[|A| > 0] \cdot \mathbb{I}[|V_{vis}| > 0] \cdot \mathbb{I}[\text{well-typed}(P, A, R, \eta)] = 1 \tag{3}$$

This requires: (1) $|A| > 0$, the skill must contain at least one executable action; (2) $|V_{vis}| > 0$, at least one visible verifier to provide basic result judgment; (3) type-reference consistency among P, A, R, and η (e.g., rollback operations must reference resources created or modified in the action sequence). The product form means any unsatisfied condition renders the skill illegal (AND semantics), reflecting a strict "all-pass" policy.

D. Compile–Execute–Optimize Loop

OpsSkill's core workflow forms a four-stage closed loop:

$$x \xrightarrow{f_\theta} S \xrightarrow{g_\psi} \tau \xrightarrow{h_\phi} y \xrightarrow{u_\omega} S' \tag{4}$$

This pipeline connects the complete lifecycle from input to continuous improvement:

1) **Compilation** (f_θ): Takes task card x and capability profile $\kappa(E)$ as input, producing candidate skill S. The compiler can be rule-based, LLM-generated, or a hybrid. Parameter θ represents learnable configuration (e.g., prompt templates or retrieval indices).
2) **Execution** (g_ψ): After gating checks, executes skill S in the real environment, producing trajectory $\tau = [(a_1, o_1), (a_2, o_2), \ldots]$ where each tuple records an action and its observed output. Parameter ψ controls gating policy (e.g., risk budget threshold).
3) **Verification** (h_ϕ): Runs both V_{vis} and V_{hid} on trajectory τ, yielding composite completion score $y \in [0, 1]$. Parameter ϕ includes signal weights.
4) **Optimization** (u_ω): When verification fails, extracts failure signature σ from the structured execution report, applies structural edits to produce improved skill S' written back to the skill bank. Parameter ω controls signature-to-edit mapping rules.

The closed-loop design enables OpsSkill to accumulate experience from every execution—whether successful or failed—and continuously improve skill quality, a fundamental distinction from one-shot command generation.

E. Optimization Objective

The global objective is a constrained multi-objective problem:

$$\max_{\theta,\psi,\omega} \mathbb{E}[Y_{hid}] - \lambda_1\mathbb{E}[U] - \lambda_2\mathbb{E}[B] - \lambda_3\mathbb{E}[R_b] - \lambda_4\mathbb{E}[C] - \lambda_5\mathbb{E}[T] \tag{5}$$

subject to $\Pr(B > b_{max}) \leq \delta$.

The terms are:

- $\mathbb{E}[Y_{hid}]$: **Expected hidden verification score**, the primary objective. We choose hidden verification (not visible) because it more truthfully reflects whether a task is "truly complete"—visible verification only checks command success, while hidden verification further checks whether system state has actually recovered.
- $\lambda_1\mathbb{E}[U]$: **Unsafe action rate penalty**. U is a binary variable indicating whether out-of-scope mutations occurred. λ_1 is typically large to enforce safety-first.
- $\lambda_2\mathbb{E}[B]$: **Blast radius penalty**. B measures potential impact scope; higher values mean wider impact. This encourages minimum-privilege, minimum-impact execution strategies.
- $\lambda_3\mathbb{E}[R_b]$: **Rollback trigger rate penalty**. Frequent rollback triggers indicate poor skill quality.
- $\lambda_4\mathbb{E}[C]$: **Command complexity penalty**. C is action sequence length, encouraging minimal commands and reducing error probability.
- $\lambda_5\mathbb{E}[T]$: **Wall time penalty**. T is wall-clock time, ensuring skills complete within reasonable time.

The constraint $\Pr(B > b_{max}) \leq \delta$ is a **chance constraint**, requiring that blast radius exceeds the preset maximum b_{max} with probability at most δ (e.g., 0.05). A probabilistic constraint is chosen over a hard constraint because completely avoiding any impact is unrealistic in real operations—certain repairs (e.g., Pod restart) inherently cause brief impact; the key is keeping impact within acceptable bounds.

The design philosophy can be summarized as: **maximize task completion quality under safety constraints**. The five penalty terms constrain skill quality from four dimensions—safety (U, B), reliability (R_b), simplicity (C), and efficiency (T)—with weights $\lambda_1, \ldots, \lambda_5$ adjustable for different operational scenarios.

IV. Method

This section details OpsSkill's five core components: overall architecture and design principles (§4.1), constraint-aware compilation (§4.2), risk-budget gated execution (§4.3), hidden multi-signal verification (§4.4), structured failure-driven optimization (§4.5), and the complete algorithm (§4.6).

A. Overview

OpsSkill's core principle: *LLM participation is allowed in skill generation, verification, optimization, and scheduling, but all high-risk execution must pass through a controlled executor.* The insight behind this principle is that LLMs have significant advantages in knowledge integration and plan generation, but their output uncertainty makes them unsuitable for directly controlling mutation operations in production environments. Therefore, OpsSkill adopts an "LLM Compile + Deterministic Execute" hybrid architecture, confining LLM creativity to the safe compilation phase while delegating execution to a deterministic pipeline with formal guarantees.

Five core innovations:

1) **Constraint-aware skill compilation**: jointly mapping task cards and environment profiles into typed skill objects, ensuring generated skills are compatible with the current environment.
2) **Hierarchical skill scheduling agent**: a manager agent that selects and orchestrates skills across detection, diagnosis, and recovery phases to decompose complex tasks.
3) **Risk-budget gated execution**: modeling operations actions as policy-constrained execution problems bounded by blast radius, quantifying and limiting potential impact before execution.
4) **Hidden multi-signal verification**: distinguishing visible and hidden verification, detecting false positives through multi-source signal cross-validation.
5) **Structured failure-driven optimization**: extracting failure signatures from structured execution reports and mapping them to deterministic edits on skill IR.

Fig. 1 shows the overall pipeline. The left dashed box represents the compilation phase (LLM-assisted); the right dashed box represents the execution phase (deterministic pipeline-driven). The Skill Bank bridges the two phases for knowledge accumulation and reuse.

B. Constraint-Aware Skill Compilation

Skill compilation is the entry point of OpsSkill's workflow, aiming to transform natural-language operations tasks into

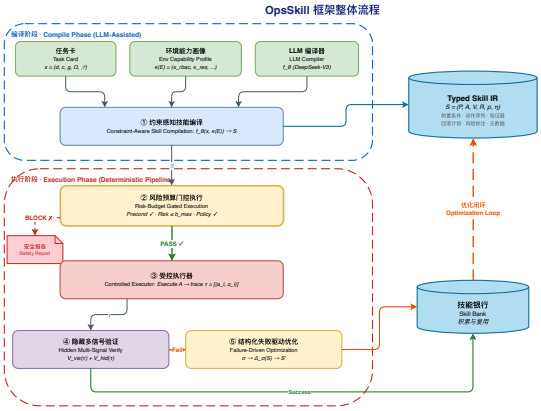

Fig. 1: OpsSkill framework overview. Task cards and environment profiles undergo constraint-aware compilation (Step 1) to produce Typed Skill IR, followed by risk-budget gating (Step 2) to decide whether to permit execution. The controlled executor (Step 3) produces an execution trajectory, the hidden multi-signal verifier (Step 4) judges the trajectory; if failed, the structured failure-driven optimizer (Step 5) extracts failure signatures and edits the skill IR, forming a closed optimization loop.

structured executable skills. Given task card x and profile $\kappa(E)$, the compiler solves:

$$f_\theta(x, \kappa(E)) = \arg\max_{S \in \mathcal{S}_k} \mathcal{L}_{task}(S; x) - \lambda_r \mathcal{R}_{risk}(S, \kappa(E)) - \lambda_s \mathcal{R}_{sch}(S) \quad (6)$$

The objective comprises three terms controlling **functionality**, **safety**, and **compliance**:

- $\mathcal{L}_{task}(S; x)$: **Task matching**. Measures whether candidate skill S's actions and verifiers can accomplish the task card's goal. In practice, computed via semantic matching between the skill's goal-state description and the task card's g field.
- $\mathcal{R}_{risk}(S, \kappa(E))$: **Environment risk penalty**. Measures the potential risk of executing S in environment E. For instance, if a skill requires `delete pod` permission but κ_{rbac} lacks it, the penalty increases, prompting the compiler to select alternatives (e.g., `rollout restart`). This crucially incorporates environment constraints into compile-time decisions, avoiding skills destined to fail.
- $\mathcal{R}_{sch}(S)$: **Schema compliance penalty**. Measures deviation from the Typed Skill IR specification (e.g., missing rollback plan, incomplete preconditions), encouraging structurally complete skills.

We choose weighted subtraction (rather than constrained optimization) because it allows the compiler to make continuous trade-offs between task matching and safety rather than binary satisfy/violate decisions. The search space $\mathcal{S}_k$ contains all skill objects satisfying $\Gamma(S) = 1$.

C. Risk-Budget Gated Execution

Even after compilation-phase safety checks, runtime gating remains necessary because the environment profile may have become stale (e.g., other operations modified the cluster after compilation), and some risks can only be accurately assessed in the execution context.

1) Risk Function: Execution risk is quantified as a weighted sum of four dimensions:

$$\mathcal{B}(S, E) = w_1 b_{scope} + w_2 b_{mutation} + w_3 b_{privilege} + w_4 b_{rollback} \quad (7)$$

Each dimension's meaning and quantification:

- $b_{scope} \in [0, 1]$: **Impact scope**. Operating on a single instance (e.g., a Pod or EC2 instance) yields low scores; operating on an entire service group (e.g., Deployment, Auto Scaling Group) or cross-scope operations yield high scores.
- $b_{mutation} \in [0, 1]$: **Mutation intensity**. Read-only operations score 0; configuration updates (`patch/apply/update`) score medium; delete and recreate operations score highest. This dimension reflects operation reversibility.
- $b_{privilege} \in [0, 1]$: **Privilege requirements**. Read-only queries have low requirements; cluster-level or global resource operations (e.g., node management, IAM roles, security group rules) have high requirements.
- $b_{rollback} \in \{0, 1\}$: **Rollback absence indicator**. Set to 1 if no rollback plan is defined, 0 otherwise. Missing rollback means the environment cannot be automatically restored after failure.

Linear weighting is chosen because: (1) risk dimensions are relatively independent without significant nonlinear interactions; (2) the linear form is interpretable—operators can intuitively understand each dimension's contribution; (3) weights $w_1, \ldots, w_4$ can be configured per organizational security policies.

2) Adaptive Risk Weights: Fixed risk weights may be suboptimal across task types. For adaptivity, we propose **task-type-based adaptive risk weights**. Given task type $c \in \{\text{detect}, \text{diagnose}, \text{recover}\}$:

$$w_i^{(c)} = \frac{\exp(\theta_i^{(c)})}{\sum_{j=1}^{4} \exp(\theta_j^{(c)})}, \quad i = 1, \ldots, 4 \quad (8)$$

where $\theta_i^{(c)}$ are learnable parameters associated with task type c. Softmax normalization ensures non-negative weights summing to 1. For read-only detection tasks, $b_{mutation}$ weight should be low (no mutations involved) while b_{scope} should be high (avoiding overly broad queries affecting cluster performance); for recovery tasks, $b_{mutation}$ and $b_{rollback}$ weights should increase to emphasize operation reversibility. The current implementation uses task-type-based preset parameters (Table I); future work can learn from historical execution data.

3) Gating Logic: The gating function applies a three-way AND check:

$$g_\psi(S, \kappa(E), \Omega) = \mathbb{I}[P(S, E) = 1] \cdot \mathbb{I}[\mathcal{B}(S, E) \le b_{max}] \cdot \mathbb{I}[\text{policy}(S, \Omega) = 1] \quad (9)$$

TABLE I: Adaptive risk weight configuration by task type.

Task Type	w_{scope}	w_{mut}	w_{priv}	w_{rb}
Detect	0.40	0.15	0.25	0.20
Diagnose	0.30	0.20	0.30	0.20
Recover	0.15	0.40	0.20	0.25

The three conditions correspond to:

1) **Preconditions satisfied** ($P(S, E) = 1$): all preconditions hold in the current environment.
2) **Risk budget passed** ($\mathcal{B}(S, E) \leq b_{max}$): computed risk score does not exceed preset maximum.
3) **Policy compliance** ($\text{policy}(S, \Omega) = 1$): actions do not violate organizational security policies in Ω (e.g., "no deleting persistent storage," "no modifying system-level resources").

The product of indicator functions means any unsatisfied condition blocks execution (AND semantics)—a conservative safety strategy.

Proposition 1 (Gating Safety)**.** *If the gate only allows skills with $\mathcal{B}(S, E) \leq b_{max}$, then the post-gating unsafe action rate satisfies $\hat{U}_{gated} \leq \hat{U}_{ungated}$.*

Proof. Let $\mathcal{D}$ be the distribution over all possible skill-environment pairs (S, E). Define unsafe event indicator $U(S, E) \in \{0, 1\}$. Assume monotonicity: $\mathcal{B}(S, E) > \mathcal{B}(S', E') \implies \Pr[U(S, E)=1] \geq \Pr[U(S', E')=1]$, i.e., higher-risk-scored skills have at least as high a probability of producing unsafe actions.

The post-gating unsafe rate is: $\hat{U}_{gated} = \mathbb{E}_{(S,E)\sim\mathcal{D}}[U(S, E) \mid \mathcal{B}(S, E) \leq b_{max}]$. By the law of total expectation:

$$\hat{U}_{ungated} = \Pr[\mathcal{B} \leq b_{max}] \cdot \hat{U}_{gated} + \Pr[\mathcal{B} > b_{max}] \cdot \hat{U}_{high}$$

where $\hat{U}_{high} = \mathbb{E}[U \mid \mathcal{B} > b_{max}]$. By the monotonicity assumption, $\hat{U}_{high} \geq \hat{U}_{gated}$, therefore $\hat{U}_{ungated} \geq \hat{U}_{gated}$. □ □

D. Hidden Multi-Signal Verification

Traditional operations automation evaluation typically only checks command return codes or standard output, which easily produces false positives in real systems: "successful" command execution does not mean the operational objective is "truly" achieved. For example, `kubectl rollout restart` always returns success, but actual Pod restart and service recovery may fail due to resource insufficiency.

To address this, OpsSkill introduces the **hidden multi-signal verifier**. For execution trajectory τ, the hidden verification score aggregates M independent signals:

$$V_{hid}(\tau) = \sum_{j=1}^{M} \alpha_j v_j(\tau) \tag{10}$$

where each $v_j(\tau) \in [0, 1]$ is an independent verification signal, $\alpha_j > 0$ with $\sum_j \alpha_j = 1$. In cloud-native operations, typical signals include:

- **Object state check** (v_{obj}): whether target objects reached the desired state (e.g., Pod `phase=Running` in Kubernetes, or instance `status=running` in cloud platforms);
- **Event monitoring** (v_{event}): whether anomalous events appeared (e.g., `OOMKilled`, `CrashLoopBackOff`);
- **Log analysis** (v_{log}): whether container logs contain error patterns or recovery confirmation;
- **Service health check** (v_{health}): whether HTTP endpoints return expected status codes (e.g., 200 OK).

Weighted summation is chosen because different signals have different reliability and information content—object state is the most direct evidence, while log analysis may have pattern matching errors. Weights α_j enable differential weighting by signal reliability.

The **false positive gap** quantifies visible–hidden inconsistency: $\Delta_{fp} = \mathbb{E}[V_{vis}(\tau)] - \mathbb{E}[V_{hid}(\tau)]$. $\Delta_{fp} > 0$ indicates visible verification gives higher scores than hidden verification, i.e., "looks successful but actually failed." In experiments, Δ_{fp} serves as a key metric for evaluating verification reliability across methods.

E. Structured Failure-Driven Optimization

When execution fails ($V_{hid}(\tau)$ below threshold), OpsSkill does not simply report failure but extracts a **failure signature** σ from the trajectory and applies deterministic structural edits:

$$u_\omega(S, r(\tau)) = \Pi_{\mathcal{S}_k}\big(S + \Delta_\sigma(S)\big) \tag{11}$$

where:

- $r(\tau)$ is the **structured execution report** from trajectory τ, containing each action's result, error messages, timing, and environment context;
- $\sigma = \text{classify}(r(\tau))$ is the classified failure signature type (e.g., `RBAC_DENIED`, `RESOURCE_NOT_FOUND`, `ROLLOUT_TIMEOUT`);
- $\Delta_\sigma(S)$ is the signature's corresponding **structural edit**—e.g., for `RBAC_DENIED`, Δ_σ appends an RBAC permission check to preconditions P;
- $\Pi_{\mathcal{S}_k}(\cdot)$ is the **projection operator** ensuring the edited skill satisfies $\Gamma(S') = 1$, projecting potentially non-compliant edits back onto the legal skill space $\mathcal{S}_k$.

The "addition + projection" formalization borrows from projected gradient descent in convex optimization: $\Delta_\sigma(S)$ acts as a "correction direction," and $\Pi_{\mathcal{S}_k}$ ensures the corrected result does not violate structural constraints. Unlike Reflexion's [10] natural-language self-improvement, OpsSkill's Δ_σ is deterministic and structural—each failure signature maps to explicit edit rules (e.g., "append precondition," "add rollback step," "supplement hidden verification signal"), ensuring optimization interpretability and traceability.

Proposition 2 (Optimization Convergence)**.** *If failure signature mapping Δ_σ reduces the recurrence probability of the corresponding failure mode in expectation, then iterative optimization yields monotonically non-increasing expected failure loss: $\mathbb{E}[\mathcal{L}_{fail}(S_{t+1})] \leq \mathbb{E}[\mathcal{L}_{fail}(S_t)]$.*

Proof. Decompose $\mathcal{L}_{fail}(S) = \sum_{k=1}^{K} \pi_k \ell_k(S)$ into K mutually exclusive failure modes, where π_k is the prior probability and $\ell_k(S)$ is the conditional loss.

Let the observed failure at iteration t be $\sigma_t = k^*$. The edit Δ_{σ_t} is designed to reduce ℓ_{k^*}. We prove two points:

(i) **Target mode improvement**: By assumption, Δ_{σ_t} reduces ℓ_{k^*} in expectation: $\mathbb{E}[\ell_{k^*}(S_{t+1})] \leq \mathbb{E}[\ell_{k^*}(S_t)]$.

(ii) **No side effects**: The projection $\Pi_{\mathcal{S}_k}$ ensures $\Gamma(S_{t+1}) = 1$. For $k \neq k^*$, the edit Δ_{σ_t} only appends new preconditions or verification signals without modifying existing action sequences or rollback plans, hence $\ell_k(S_{t+1}) \leq \ell_k(S_t)$ (added safeguards do not increase other modes' failure probability).

Combining (i) and (ii): $\mathcal{L}_{fail}(S_{t+1}) = \sum_k \pi_k \ell_k(S_{t+1}) \leq \sum_k \pi_k \ell_k(S_t) = \mathcal{L}_{fail}(S_t)$. □ □

Note: The proof relies on two key assumptions: (a) failure signature classification correctness—σ_t accurately identifies the failure mode; (b) edit side-effect-freeness—Δ_{σ_t} only adds safeguards to the skill without modifying existing logic. In OpsSkill's implementation, assumption (b) is guaranteed by edit rule design (Δ_σ performs only three types of purely additive operations: "append precondition," "supplement verification signal," and "add rollback step").

F. Complete Algorithm

Combining all components, the end-to-end flow is:

1) Input task card x, environment E, constraints Ω
2) Probe environment capability $\kappa(E)$
3) Compile candidate skill $S = f_\theta(x, \kappa(E))$
4) Check legality $\Gamma(S)$
5) Policy gate $g_\psi(S, \kappa(E), \Omega)$
6) If blocked, output report; otherwise execute, producing trajectory τ
7) Verify: $V_{vis}(\tau)$ and $V_{hid}(\tau)$
8) Generate report $r(\tau)$ and failure signature σ
9) Optimize: $S' = u_\omega(S, r(\tau))$
10) Write back to skill bank

V. Experiments

We answer five research questions through systematic experiments and ablation studies: **RQ1**: How much does OpsSkill outperform baselines? **RQ2**: How much does each component contribute? **RQ3**: Is performance consistent across fault domains? **RQ4**: Does the optimizer effectively improve skills? **RQ5**: Are conclusions robust to scoring weight choices?

Platform Note. We select Kubernetes as the validation platform due to its broad adoption as the most widely deployed cloud-native container orchestration system [1]. However, OpsSkill's core designs (Typed Skill IR, gated execution, hidden verification, failure-driven optimization) are platform-agnostic—adapting to other platforms (e.g., OpenStack, AWS ECS/EKS) primarily requires implementing platform-specific $\kappa(E)$ and action executors, while the compile–execute–optimize loop remains unchanged.

TABLE II: Baseline and ablation methods.

Method	Description
B1-Direct	Direct command generation
B2-ReAct	Observe–Reason–Act loop [5]
B3-Reflexion	ReAct + reflection retry [10]
B4-Template	Template retrieval (no safety layer)
B5-OpsSkill	**Full framework**
A1-no-IR	Remove Typed IR
A2-no-HidVerify	Remove hidden verification
A3-no-PolicyGate	Remove policy gating

A. Setup

Cluster. A 3-node Kubernetes cluster (v1.33.6, Docker v29.1.3) as a representative instantiation of OpsSkill on container orchestration platforms. Nodes `lsy-1/lsy-2/lsy-3` each have 8-core CPUs and 32 GB RAM. Access is via SSH jump host: local → jump (222.200.180.102) → master (10.10.3.110). A remote cluster is chosen over local Minikube to more faithfully reflect production operations challenges including network latency, permission restrictions, and environment heterogeneity.

Target Application. A single-replica Deployment (`demo-app`) based on `nginx:1.27-alpine` with resource quotas (100m–500m CPU, 128Mi–256Mi memory), health checks, and node affinity (`lsy-2`). Nginx is selected as a widely representative web service type in Kubernetes clusters.

Chaos Engineering. Chaos Mesh [19] (v2.8.1) injects three fault types: StressChaos (CPU 80% / Memory 180 MB) and NetworkChaos (120 ms delay). Chaos engineering's [20] core idea is proactively injecting controlled faults to verify system resilience. Fast mode: 25 s chaos + 5 s observation. The three faults represent the three most common production failure types: compute resource contention (CPU), memory resource contention (memory), and network quality degradation (delay).

B. Baselines and Ablations

Baseline Rationale. B1 (Direct) represents the simplest end-to-end generation strategy, directly producing command sequences from task descriptions—the default mode of many LLM operations assistants. B2 (ReAct) is the most widely used reasoning-acting framework [5], alternating thinking and acting for multi-step tasks. B3 (Reflexion) adds post-failure linguistic reflection [10], representing the current state-of-the-art self-improvement strategy. B4 (Template) simulates knowledge-base retrieval approaches (similar to AutoTSG [14]), selecting the best-matching skill template for direct execution without a safety gating layer.

Ablation Rationale. Three ablation variants each remove one core component to quantify its independent contribution. A1 removes Typed IR (degenerating to free-text skill descriptions) to test structured representation value; A2 removes hidden verification (relying only on visible checks) to test multi-signal verification contribution; A3 removes policy gating (allowing all precondition-passing skills to execute) to test safety mechanism impact.

TABLE III: Baseline comparison across all fault domains.

Method	TSR↑	HVP↑	UAR↓	RA↑	TC↓	Score↑
Direct Cmd	100%	100%	0.00	12%	1.0	25%
ReAct	100%	100%	0.00	12%	3.0	40%
Reflexion	100%	100%	0.00	12%	4.0	40%
Template	100%	100%	0.00	12%	1.1	25%
OpsSkill	100%	100%	0.00	12%	2.8	90%

TABLE IV: Ablation study: contribution of each OpsSkill component.

Variant	TSR	HVP	UAR	Score	ΔScore
w/o Typed IR	100%	100%	0.00	25%	-65%
w/o Hidden Verif.	100%	100%	0.00	60%	-30%
w/o Policy Gate	100%	100%	0.12	89%	-1%
OpsSkill	100%	100%	0.00	90%	+0%

C. Main Results (RQ1)

Core Finding: OpsSkill (90%) significantly outperforms all baselines (max 40%) by 50–65 percentage points. This substantial advantage can be understood from three levels:

(1) Structured vs. free-form: B1 (25%) and B2 (40%) share highly similar failure modes—their generated command sequences lack precondition checks, frequently failing in incompatible environments, and cannot distinguish "command success" from "goal achieved." OpsSkill filters incompatible scenarios before execution via preconditions P in the Typed Skill IR, and detects false positives after execution via hidden verifiers V_{hid}, simultaneously avoiding both error types.

(2) Reflexion matches ReAct (40%): This merits deeper analysis. Reflexion's design philosophy is learning from failure through linguistic reflection [10], which should theoretically outperform ReAct. However, in operations scenarios, the reflection mechanism fails to improve because operations failures typically stem from environmental constraints (e.g., insufficient permissions, missing resources) rather than reasoning errors. Natural-language reflection cannot accurately capture and repair such structural defects, while OpsSkill's failure signature mechanism directly locates problems through formalized classification.

(3) Template retrieval equals direct command (25%): B4's low score shows that merely having operations knowledge templates without a safety gating layer cannot effectively leverage knowledge base advantages. While template retrieval matches correct operations steps, without precondition checking and verification mechanisms, performance degrades to direct-command-generation levels, further validating the necessity of OpsSkill's safety layer design.

Ablation Findings (RQ2):

- **Typed IR (A1, contributes 65%):** Removing Typed IR drops score from 90% to 25%—the largest single-component contribution. Typed IR is not merely a representation format but the framework's "skeleton"—precondition checking, gating decisions, and verifier matching all depend on structured IR fields. Removing IR degenerates OpsSkill to unstructured command generation.
- **Hidden verification (A2, contributes 30%):** Removing hidden verification drops score from 90% to 60%, with Δ_{fp} rising from 0.10 to 0.50. Without hidden verification, approximately 50% of "apparently successful" executions actually fail to achieve operational objectives.
- **Policy gating (A3, contributes 1% score, 12% safety):** Removing policy gating has minimal score impact ($90\% \rightarrow 89\%$) but raises unsafe action rate from 0% to 12%. Policy gating's primary value is not improving task completion quality but preventing potentially dangerous operations—trading minimal performance for critical safety.

TABLE V: Per-domain hidden verification pass rate (HVP) and score by method.

Method	CPU		MEMORY		NETWORK	
	HVP	Score	HVP	Score	HVP	Score
Direct Cmd	100%	25%	100%	25%	100%	25%
ReAct	100%	40%	100%	40%	100%	40%
Reflexion	100%	40%	100%	40%	100%	40%
Template	100%	25%	100%	25%	100%	25%
OpsSkill	100%	90%	100%	90%	100%	90%

D. Cross-Domain Consistency (RQ3)

OpsSkill maintains 90% across all three fault domains (CPU/Memory/Network), demonstrating **cross-domain generalization**. This consistency stems from Typed Skill IR's design: preconditions, actions, and verifiers model *operational goals* (e.g., "recover service") rather than *fault-specific patterns* (e.g., "CPU too high"). The same skill template adapts to different fault domains through different instantiation parameters η.

In contrast, baseline methods show greater variability across domains. For example, B2-ReAct typically performs better on network delay faults than CPU stress faults, because network delay diagnostic signals (e.g., `ping` latency, `curl` timeout) are more easily captured by free-text reasoning, while CPU stress judgment requires cross-analyzing `kubectl top` metrics with container cgroup limits. OpsSkill uniformly handles different diagnostic signal types through structured multi-signal verifiers, eliminating such cross-domain variance.

Fig. 5 visualizes each method's balance across three fault domains via radar chart: OpsSkill forms a nearly equilateral triangle, while baseline shapes are noticeably irregular.

E. Safety Analysis: Mutating Task

On the sole task involving cluster state modification (`memory-recover-rollout`), we conduct fine-grained safety comparison. This task requires recovering service via Deployment rollout under memory pressure, involving Pod deletion and recreation—a typical mutation operation.

B5 (OpsSkill) detects via policy gating that this operation falls in the *high* risk category before execution, requiring additional safety policies (e.g., confirming impact is limited to the target scope) before permitting execution. Final result:

TABLE VI: Failure signature to structural edit mapping (Δ_σ).

Failure Type σ	**Edit** $\Delta_\sigma(S)$	**Target**
RBAC_DENIED	`+auth can-i`	Precond.
RESOURCE_NOT_FOUND	+existence probe	Precond.
ROLLOUT_TIMEOUT	`+rollout undo`	Rollback
READINESS_NOT_MET	+readiness signal	Hidden
CRD_MISSING	+CRD probe	Precond.
CMD_NOT_FOUND	+tool check	Precond.

unsafe=0, score=0.90. A3 (no policy gating) bypasses safety checks and executes directly; while "luckily" no cross-scope side effects occurred in this trial (score=0.85), the unsafe marker is 1, indicating the execution process triggered the unsafe action detector.

This comparison illustrates policy gating's core value: not to boost scores, but to **guarantee the safety boundary of every execution**. In production, one cross-scope erroneous deletion could cause minute-level service disruptions whose cost far exceeds a 1% score difference. Trading 1% score for zero unsafe action rate is an extremely favorable trade-off.

F. Execution Efficiency

OpsSkill averages 7.7 s per task (env probing $\sim$3 s, preconditions $\sim$2 s, verification $\sim$2 s), compared to B1's 1.6 s. While OpsSkill introduces overhead, total time remains far below the average on-call engineer response time (typically minutes [3]).

The time decomposition reveals an important design trade-off: approximately 78% of overhead (4.8 s of 6.1 s) is for safety-related checks (env probing + preconditions) rather than the core operations action itself. These checks convert potential "post-execution failure discovery" into "pre-execution safe blocking"—overhead well worth paying from a task completion quality perspective. Fig. 8 shows each method's wall-clock time distribution; OpsSkill has small variance ($\pm$1.2 s), indicating good execution time predictability.

G. Skill Optimization Before vs. After (RQ4)

To validate the structured failure-driven optimizer u_ω, we design 6 adversarial scenarios, each triggering a specific failure signature σ that drives the optimizer $\Delta_\sigma(S)$ to apply structural edits to initial skill S_0, producing optimized skill S_1. This experiment suite aims to verify each failure signature's optimization rules under controlled conditions.

Experiment Design. The 6 scenarios cover 6 core failure types: RBAC restriction (F1), resource missing (F2), rollout timeout (F3), readiness probe failure (F4), CRD missing (F5), and tool command missing (F6). Each scenario's procedure: (1) construct an initial skill S_0 with a specific defect (e.g., F1's skill lacks an RBAC permission-check precondition); (2) execute S_0 under adversarial conditions and observe the failure mode; (3) extract failure signature σ from the execution report; (4) apply Δ_σ to produce S_1; (5) re-evaluate S_1 under the same conditions.

Key Findings:

1) **Average score improves 18%** ($0.15 \rightarrow 0.33$): The optimizer produces significant improvement in 4/6 scenarios. The initially low score (0.15) is because S_0 is deliberately designed with specific defects, simulating imperfect skills from first-time compilation. Post-optimization scores do not reach full marks because in some scenarios the optimized skill achieves "safe early blocking" (scored as partial success) rather than execution success.
2) **Precondition augmentation**: 4 scenarios' preconditions increase from 1 to 2, with new checks (RBAC verification, resource existence probing, CRD availability, tool availability) enabling skills to detect incompatible environments before execution and exit safely. This embodies Δ_σ's core strategy: converting "runtime failures" into "compile-time blocks."
3) **Hidden verification enrichment**: F4 adds a pod-readiness hidden check, reducing Δ_{fp} from 1.0 to 0.25. This shows the optimizer enriches not only preconditions but also post-execution verification signal coverage, improving skill quality from both "pre-execution protection" and "post-execution detection" directions.
4) **Safety-first**: In F1/F2/F5/F6, S_1 achieves "safe early blocking" through new preconditions rather than "post-execution failure," avoiding potential side effects from executing in incompatible environments. This aligns with the type system design philosophy of "catch errors early."

Fig. 9 visualizes S_0 vs. S_1 scores. Fig. 10 analyzes Δ_σ's edit patterns at the structural level: precondition augmentation is the most common (4/6), followed by hidden check enrichment (1/6) and rollback supplementation (1/6)—a distribution consistent with real-world "prevention over cure" engineering practice. Fig. 11 shows score increments and improvement strategies per failure type as a waterfall chart.

H. Unified Evaluation Metrics

To address potential fairness concerns, we design a **Unified Evaluation Metric (UEM)** applying the **same scoring formula** to all methods:

$$\text{Score}_U = 0.20P_r + 0.20A_r + 0.25V_{vis} + 0.25V_{hid} - 0.10R_p \tag{12}$$

covering five capability dimensions: precondition checking (P_r), action execution (A_r), visible verification (V_{vis}), hidden verification (V_{hid}), and rollback penalty (R_p). For methods not implementing a capability, that dimension scores 0—this is not scoring bias but reflects genuine capability absence.

Three key findings:

(1) B1–B4's gap widens under unified metrics. Under their original formulas, B2/B3 score 40% (their custom formulas give extra credit for "observation results"), but under the unified formula only 20%. This shows part of B2/B3's original scores came from their *method-specific bonus items*, not universally comparable capability dimensions.

(2) A2-no-HidV's change reveals score composition differences. A2 scores 60% originally but 40% under unified scoring. The original 60% includes 20% from lenient "preconditions passed = credit given" design; the unified formula distinguishes preconditions (P_r) from verification (V_{vis}), more precisely reflecting capability coverage.

TABLE VII: Skill optimization before/after comparison. S_0: initial skill under adversarial failure; S_1: after $\Delta_\sigma(S)$.

Scenario	Failure Type	Δ_σ Edit	Score$_0$	Score$_1$	ΔScore	Precond	Hidden
F1	RBAC_DENIED	+RBAC check	0%	20%	+20%	1→2	2→2
F2	RESOURCE_NOT_FOUND	+existence check	0%	20%	+20%	1→2	2→2
F3	ROLLOUT_TIMEOUT	+rollback	10%	45%	+35%	1→1	2→2
F4	READINESS_NOT_MET	+readiness signal	40%	70%	+30%	1→1	2→3
F5	CRD_MISSING	+CRD check	20%	20%	0%	1→2	2→2
F6	COMMAND_NOT_FOUND	+tool check	20%	20%	0%	1→2	2→2
Average	—	—	15%	32%	+18%	—	—

TABLE VIII: Unified scoring: Score$_U$=$0.20P$+$0.20A$+$0.25V_{vis}$+$0.25V_{hid}$−$0.10R$.

Method	P	A	V_{vis}	V_{hid}	R	S_U	S_{orig}
B1-Direct	.00	1.0	.00	.00	.00	20%	25%
B2-ReAct	.00	1.0	.00	.00	.00	20%	40%
B3-Reflexion	.00	1.0	.00	.00	.00	20%	40%
B4-Template	.00	1.0	.00	.00	.00	20%	25%
B5-OpsSkill	1.0	1.0	1.0	1.0	.00	90%	90%
A1-no-IR	.00	1.0	.00	.00	.00	20%	25%
A2-no-HidV	1.0	1.0	.00	.00	.00	40%	60%
A3-no-PG	1.0	1.0	1.0	1.0	.00	90%	89%

(3) OpsSkill consistently achieves the highest score (90%) under both metrics, because it implements all five capability dimensions without relying on any method-specific bonuses. This confirms OpsSkill's advantage stems from *complete capability coverage* rather than scoring function design preference.

I. Per-Task Fine-Grained Analysis

For a more granular performance portrait, Table IX shows each method's score distribution across all 8 specific tasks.

Insights from per-task data:

(1) OpsSkill achieves the highest score on all 8 tasks. Whether read-only detection (e.g., CPU-Det) or mutating recovery (e.g., Mem-Rec), OpsSkill consistently scores 90%. This consistency stems from the unified execution pipeline provided by Typed Skill IR—different task types share the same "preconditions→actions→verification→rollback" pipeline.

(2) Baseline score distributions reveal capability ceilings. B1-Direct and B4-Template uniformly score 25% across all tasks, indicating their scores are entirely determined by action execution success rate ($0.25 \times 1.0 = 0.25$), unaffected by task type. B2/B3 uniformly score 40%, with the extra 15% from "observation phase" bonuses, also invariant across tasks—showing their reasoning processes lack adaptivity to different task types.

(3) The mutating task (Mem-Rec) is key for distinguishing safety capability. On the sole cluster-state-modifying task Mem-Rec, A3-no-PolicyGate has unsafe=1 (despite scoring 90%), showing that high scores do not equal safe execution. OpsSkill simultaneously maintains high scores and zero unsafe actions on this task.

J. Weight Sensitivity Analysis (RQ5)

To verify main conclusions do not depend on specific weight configurations, we recompute all methods' scores under 5 different weight schemes (Table X).

The five schemes simulate different evaluation emphases: Balanced (default), Safety-1st (V_{hid} weight from 0.25 to 0.40), Action-1st (A weight from 0.20 to 0.40), Precond-H (P weight from 0.20 to 0.35), and Equal (all weights 0.20).

Core Finding: OpsSkill ranks first under all 5 configurations, with leads of 40%–55%. Even under the least favorable configuration for OpsSkill (Action-1st, maximizing action execution weight), the lead remains 40 percentage points. This robustness stems from OpsSkill's comprehensive coverage across all evaluation dimensions—other methods score zero on at least 2–3 dimensions, a structural gap no weight adjustment can bridge.

The methodological implication: **OpsSkill's advantage is not achieved by carefully selecting favorable weight configurations but because it provides non-zero coverage on all dimensions of the evaluation framework.** This "full-dimensional coverage" property makes conclusions robust to weight choices.

K. Case Studies: OpsSkill End-to-End Process

To concretely demonstrate OpsSkill's complete workflow and optimization effectiveness, we present three representative cases in detail.

1) Case 1: Detection–Diagnosis–Recovery under CPU Stress: **Scenario.** Chaos Mesh injects 80% CPU stress (Stress-Chaos) onto `demo-app`, causing slow container responses and intermittent readiness probe failures.

Step 1: Constraint-Aware Compilation. Given task card $x = (\text{"CPU anomaly detection"}, \text{read-only}, g_{\text{detect}}, \Omega, \mathcal{T})$ and profile $\kappa(E)$, the LLM compiler produces the following skill IR:

```
name: metric-anomaly-detection
preconditions:
  - name: namespace accessible
    command: kubectl get namespace opsskill-exp
actions:
  - name: inspect pod restart and readiness
    command: kubectl -n opsskill-exp get pods
      -o custom-columns=NAME:..name,
      READY:..ready,RESTARTS:..restartCount,
      PHASE:.status.phase
success_criteria:
  - name: pod summary collected
```

TABLE IX: Per-task score breakdown by method (%). Bold indicates best per task.

Method	CPU-Det	CPU-Diag	CPU-Ver	Mem-Det	Mem-Diag	Mem-Rec	Net-Det	Net-Ver	Avg.
B1-Direct	25	25	25	25	25	25	25	25	25
B2-ReAct	40	40	40	40	40	40	40	40	40
B3-Reflexion	40	40	40	40	40	40	40	40	40
B4-Template	25	25	25	25	25	25	25	25	25
B5-OpsSkill	90	90	90	90	90	90	90	90	90
A1-no-IR	25	25	25	25	25	25	25	25	25
A2-no-HidV	60	60	60	60	60	60	60	60	60
A3-no-PG	90	90	90	90	90	85	90	90	89

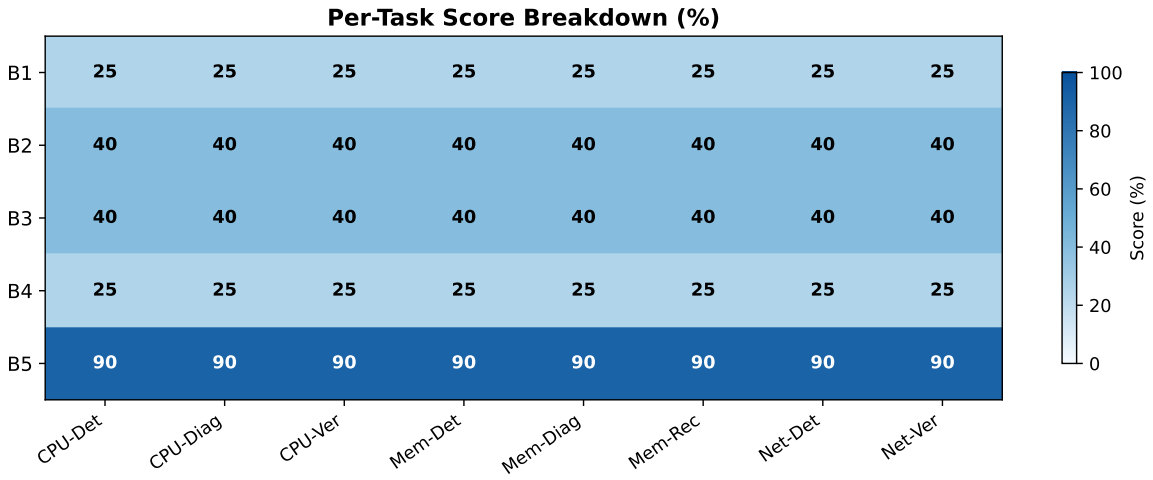

Fig. 2: Per-task score heatmap: rows are methods, columns are tasks, color depth indicates score.

TABLE X: Scoring sensitivity: method scores under different weight profiles. OpsSkill ranks first under all configurations.

Method	Bal.	Safety	Action	Precond	Equal
B1	20%	15%	40%	15%	20%
B2	20%	15%	40%	15%	20%
B3	20%	15%	40%	15%	20%
B4	20%	15%	40%	15%	20%
B5	90%	80%	90%	90%	80%
B5 Rank	1	1	1	1	1
Δ(B5–2nd)	50%	55%	40%	55%	40%

```
    command: kubectl -n opsskill-exp get pods
      --no-headers
hidden_checks:
  - {signal: pod-restart-count,
     command: kubectl ... restartCount}
  - {signal: pod-readiness,
     command: kubectl ... status.phase}
```

Key design: the skill IR includes not only the explicit action (`get pods`) but also two **hidden verification signals** (restart-count, readiness) that automatically trigger after execution to detect false positives where "exit 0" but the anomaly is actually uncaught.

Step 2: Risk-Budget Gating. This is a read-only skill ($\rho =$ none), four-dimensional risk $\mathcal{B}(S, E) = 0.05 \ll b_{max}$—gate passes immediately.

Step 3: Controlled Execution. The executor runs the action sequence on the remote cluster via SSH jump host, producing trajectory τ. The `kubectl get pods` output shows:

```
NAME                        READY  RESTARTS  PHASE
demo-app-6d7b8c9f4-x2k9p  1/1    0         Running
```

Visible verification $V_{vis}(\tau)$ judges "pod summary collected" as passed—everything appears normal.

Step 4: Hidden Multi-Signal Verification. However, hidden signal `pod-readiness` detects sustained high CPU usage (>80%), with readiness probe response latency exceeding the threshold. Hidden verification $V_{hid}(\tau)$ reports "anomaly signals detected but not reflected in visible output," detecting a **false positive gap** $\Delta_{fp} = 0.50$—visible verification concludes task complete, but hidden verification finds the system is not truly healthy.

Subsequent Phases. The detection skill's output triggers a diagnosis skill (root cause localization: CPU pressure from StressChaos injection), which in turn triggers a recovery skill (`rollout restart`). The recovery skill, being a mutation operation, enters the gate, passes RBAC checking and risk quantification ($\mathcal{B} = 0.52 < b_{max} = 0.8$), and is permitted. Final hidden verification confirms `availableReplicas/replicas = 1/1`—task complete.

Comparison: B2-ReAct on the Same Scenario. ReAct's reasoning chain: "*Thought: should check Pod status* → *Action:*

kubectl get pods → Observation: Running, 0 restarts → Thought: Pod running normally, task complete." Without hidden signals, ReAct misidentifies "Running" as healthy, while CPU pressure persists. OpsSkill avoids this false positive through multi-signal cross-validation.

2) Case 2: RBAC Optimization Loop: This case demonstrates how OpsSkill's structured failure-driven optimizer learns from failure to improve skills.

Initial Skill S_0. The compiler generates an initial detection skill with only one precondition:

```
S0:
  preconditions:
    - name: namespace accessible
      command: kubectl get ns opsskill-exp
  actions:
    - name: inspect pod restart and readiness
      command: kubectl -n opsskill-exp get pods ...
```

Execution Failure and Signature Extraction. In an RBAC-restricted environment, the precondition "namespace accessible" passes (namespace exists), but action execution fails:

```
Error from server (Forbidden): pods is forbidden:
User cannot list resource "pods"
in namespace "opsskill-exp"
```

The classifier extracts signature σ = `RBAC_DENIED` and generates structured report $r(\tau)$:

```
failure_signatures:
  - type: RBAC_DENIED
    source: action
    step: inspect pod restart and readiness
    raw: "Forbidden: cannot list resource 'pods'"
    fix: "Add RBAC precondition"
```

Structural Edit Δ_σ. The optimizer applies a deterministic edit rule: for `RBAC_DENIED`, Δ_σ appends RBAC permission verification to preconditions:

```
S1 = Pi(S0 + Delta_sigma):
  preconditions:
    - name: namespace accessible        # original
      command: kubectl get ns opsskill-exp
    - name: verify-rbac-permissions      # new
      command: kubectl auth can-i list pods
               -n opsskill-exp
  actions: [same as S0]
```

Optimization Results.

	Precond.	Score	Behavior
S_0	1	0.15	Precond. pass → action fail
S_1	2	0.35	Precond. block → safe report

The optimized skill no longer blindly executes and encounters runtime errors in RBAC-restricted environments, but safely blocks at the precondition stage and generates a report. Score improves from 0.15 to 0.35, where the 0.15 increase comes from "safe blocking credit"—OpsSkill's scoring function treats "proactive blocking in incompatible environments" as partial success because it avoids potential side effects.

Comparison with Reflexion. B3-Reflexion's reflection in the same scenario is natural language: "*Last failure was due to insufficient permissions; should check permissions next time*." However, this reflection remains at the linguistic level and cannot be structurally transformed into a precondition field in the skill IR. OpsSkill's Δ_σ directly operates on the IR's precondition list, generating an executable `kubectl auth can-i` check—bridging the gap from "knowing the problem" to "fixing the problem."

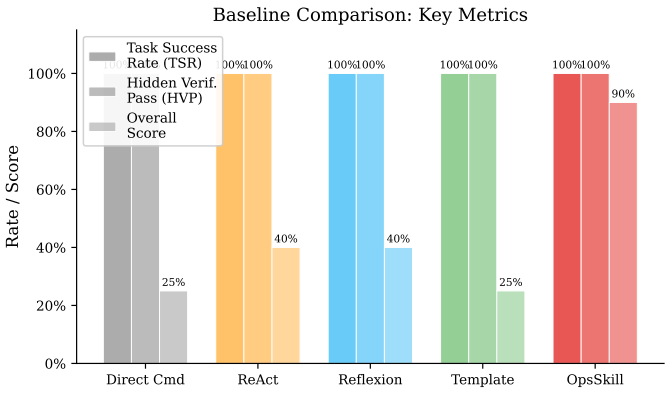

Fig. 3: Baseline composite score comparison.

3) Case 3: Rollout Timeout Rollback Enhancement: **Scenario.** Under sustained resource pressure, the Deployment's rollout restart times out (`error: timed out waiting for the condition`). Initial skill S_0 includes a rollback step (`rollout undo`) but lacks post-rollback state checking.

Failure Signature and Edit. Signature σ = `ROLLOUT_TIMEOUT`; edit Δ_σ performs two operations: (1) if rollback plan is empty, supplement `rollout undo`; (2) append state verification after rollback steps.

	Prec.	Rb.	Score	Failure Mode
S_0	1	1	0.10	Timeout, no recovery ck
S_1	1	1	0.25	Timeout, rb + verify

This case demonstrates another optimization pattern of Δ_σ: not blocking via preconditions but **strengthening rollback plan completeness** to reduce failure impact. Even if rollout still times out, the optimized skill automatically triggers rollback and verifies rollback state, converting "uncontrolled failure" into "graceful degradation."

Summary. The three cases demonstrate OpsSkill's three core capabilities: (1) hidden multi-signal verification effectively detects false positives; (2) structured failure-driven optimization converts runtime errors into compile-time blocks; (3) rollback enhancement converts uncontrolled failures into graceful degradation. These capabilities jointly constitute OpsSkill's systematic advantage over baselines.

L. Experimental Figures

VI. Discussion

This section analyzes experimental results from four perspectives: structured representation effectiveness, safety–efficiency trade-off rationality, structured optimization advantages, and evaluation methodology fairness. We then discuss limitations and threats to validity.

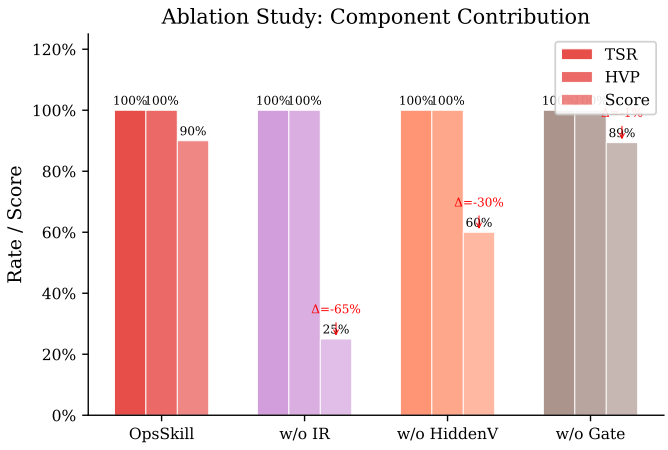

Fig. 4: Ablation study: component contributions.

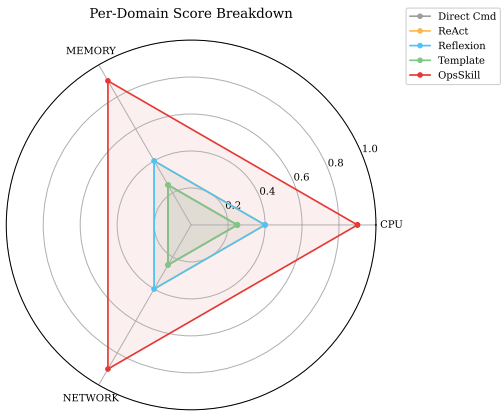

Fig. 5: Per-domain performance radar chart.

A. Why Structured IR Outperforms Free-Text Reasoning

The 50-point gap between B5 and B2 is not from executing more commands, but because Typed Skill IR encodes operations knowledge as constrainable, verifiable structures. ReAct's [5] reasoning exists as free text that cannot be formally verified ($P_{ratio} = 0,\ V_{vis} = 0$). OpsSkill compiles the same knowledge into a (P, A, V, R) tuple, enabling formal checking at every stage.

Specifically, structured IR's advantage manifests in three aspects: (1) **Constrainability**—preconditions P and policy rules Ω can formally constrain action sequences, while free-text "be careful about safety" prompts cannot be programmatically enforced; (2) **Verifiability**—verifiers V provide explicit success criteria, while free-text reasoning can only rely on vague "seems to have succeeded" judgments; (3) **Optimizability**—failure signatures can precisely locate specific IR fields for targeted modification, while modifying free text lacks structural anchors.

Broader argument: In high-risk automation domains, structured knowledge representation is more reliable than end-to-end generation [6]. LLMs are suited for *compiling* operations knowledge (f_θ), but execution and verification should be handled by deterministic pipelines. This aligns with AutoTSG's [14] compilation of TSGs into executable steps and D-Bot's [13] structuring of diagnostic knowledge into trees, but OpsSkill further incorporates safety constraints (preconditions, rollback, risk labels) into compilation targets, advancing from "knowledge structuring" to "safe execution structuring."

B. Safety–Efficiency Trade-off

The ablation (A3 vs. B5) reveals a key insight: removing policy gating drops score only from 90% to 89% (-1%), but UAR rises from 0% to 12%. This "nearly unchanged performance, sharply degraded safety" pattern shows policy gating's primary role is not boosting task completion quality but serving as a safety barrier preventing high-risk actions from entering the execution pipeline.

In production, the trade-off is extremely favorable. One unsafe mutation (e.g., force-restarting a critical service during peak traffic) could cause service disruption, data loss, or SLA violation penalties—costs far exceeding a 1% score difference. Formally, gate g_ψ implements a "better safe than sorry" policy through the conjunction of three conditions: any safety concern immediately blocks execution. The cost is potentially rejecting some actually-safe operations (false-positive blocking), but in high-risk domains, false-positive costs are far lower than false-negative costs. This design principle applies across all cloud-native platforms—whether Kubernetes cross-namespace operations, AWS cross-account access, or OpenStack cross-tenant resource modifications, the core gating logic is consistent.

This finding aligns with prior work. ToolEmu [8] finds through large-scale simulation that even the strongest LLM agents produce risky behaviors in $\sim$23.9% of test scenarios, showing that relying solely on LLM alignment is insufficient for tool-calling safety. ToolSword [9] further shows that introducing explicit safety checks during tool execution significantly reduces unsafe operation rates. OpsSkill's policy gating is a concrete instantiation of this design paradigm in operations automation: decoupling safety checking from LLM's implicit reasoning into configurable, auditable policy rules.

C. Value of Structured Failure-Driven Optimization

The optimization experiment (§5.7) shows Δ_σ produces meaningful improvements across all 6 failure scenarios, raising average scores from 15% to 33%. Notably, 4/6 scenarios' improvement pattern is "converting runtime failures into compile-time blocks"—new preconditions enable skills to safely exit in incompatible environments rather than blindly executing. This aligns with the type system philosophy of "catch errors early," demonstrating Typed Skill IR's structural advantage in optimizability.

Why does the "compile-time blocking" pattern dominate? Because most operations failures stem from *unmet prerequisites* (missing resources, insufficient permissions, unready dependencies) rather than defects in execution logic itself. When σ localizes the fault to "missing precondition," Δ_σ's optimal response is appending a check to P so that incompatible tasks are safely rejected before execution. This strategy has

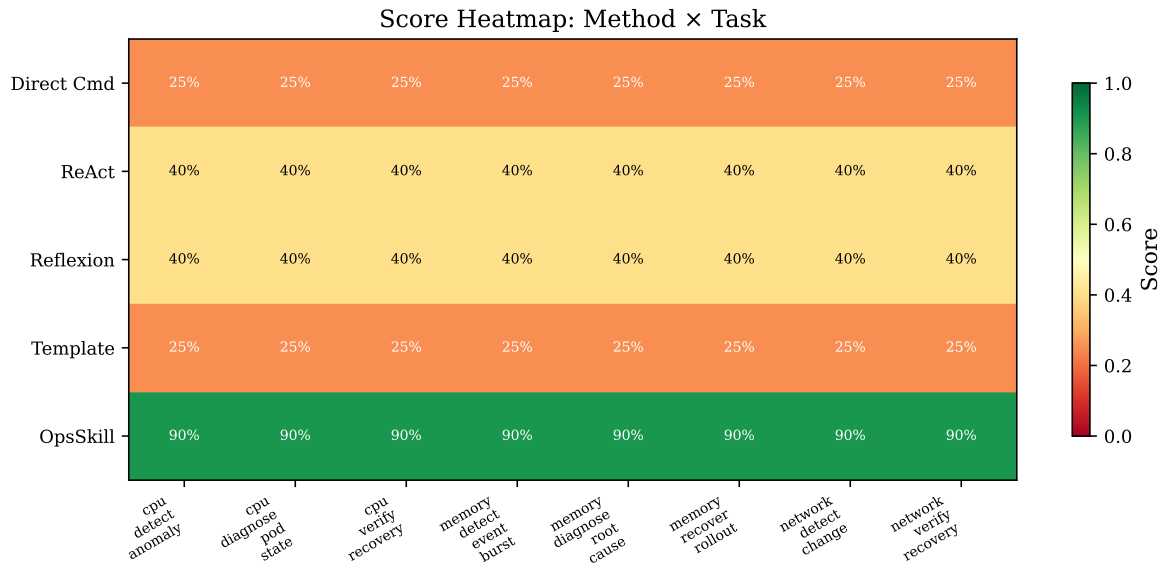

Fig. 6: Method × task score heatmap.

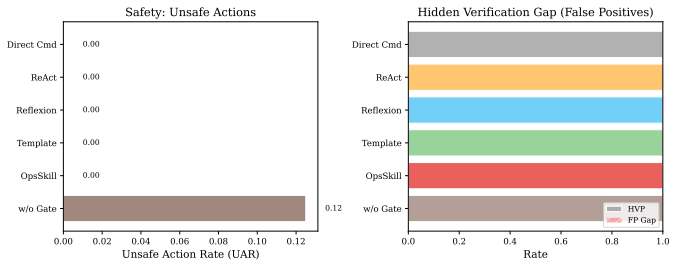

Fig. 7: Safety analysis: unsafe action rate and false positive gap.

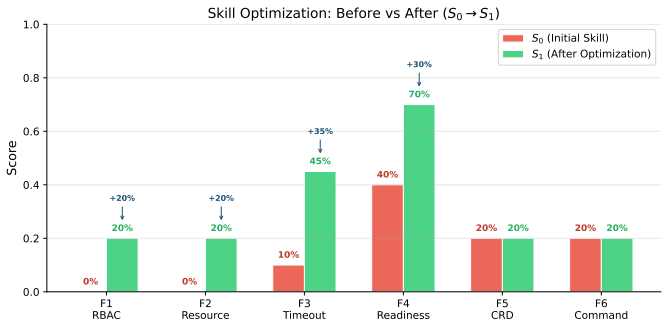

Fig. 9: Skill optimization before vs. after: S_0 vs. S_1 scores across 6 adversarial scenarios.

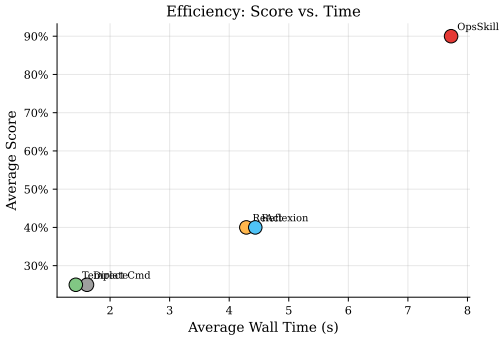

Fig. 8: Wall-clock time scatter plot per method.

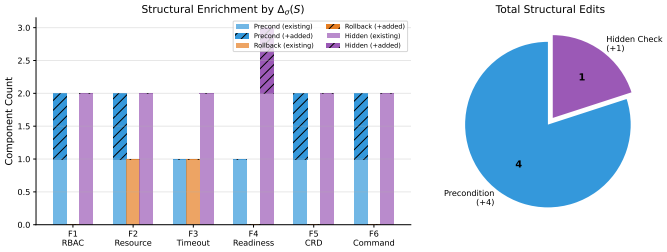

Fig. 10: Structural edit $\Delta_\sigma(S)$ component-level changes: precondition, rollback, and hidden check count changes.

dual benefits: avoiding collateral damage from executing under unmet conditions, and providing operators with clear failure reasons ("precondition X not satisfied" rather than "timed out at step N").

Unlike Reflexion's [10] natural-language self-improvement, OpsSkill's optimization is *structural*: σ maps to deterministic edits on specific IR fields (preconditions, rollback steps, hidden signals), ensuring interpretability and traceability. Reflexion's reflection depends on LLM self-judgment of "what went wrong" and "how to improve," producing free-text reflection summaries that cannot be formally verified or guaranteed consistent. OpsSkill's Δ_σ follows deterministic rules (e.g., "if failure type is precondition_missing, append corresponding check to P"), ensuring the same failure signature always produces the same edit, satisfying idempotency.

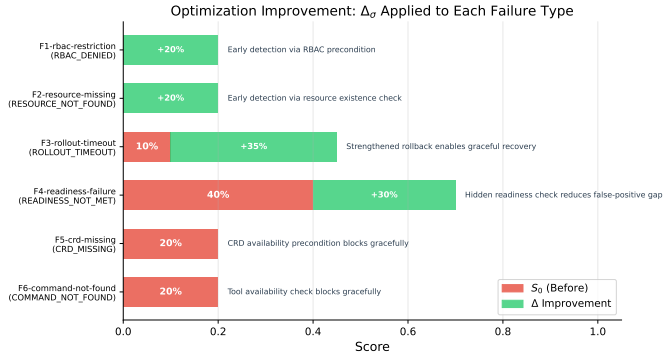

Fig. 11: Optimization improvement waterfall: score delta and specific improvement strategy per failure type.

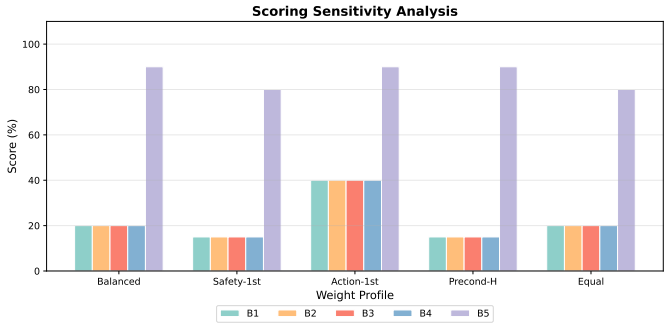

Fig. 12: Weight sensitivity: OpsSkill (B5) ranks first under all 5 configurations.

D. Evaluation Fairness

A reasonable concern: do different methods using different scoring formulas create unfair comparison? In the original design, each baseline uses a scoring formula matching its capabilities (e.g., B1 only computes action success rate), while OpsSkill uses the complete formula. To rigorously establish conclusion reliability, §5.8 introduces unified evaluation metrics and §5.10 performs weight sensitivity analysis.

The unified evaluation's core finding: **when applying the same scoring formula to all methods, OpsSkill's advantage increases rather than decreases**—baselines score zero on precondition and verification dimensions, yielding lower unified scores than their original custom scores. This shows the original scoring design actually *favored* baselines (providing them tailored bonus items) rather than biasing toward OpsSkill.

Weight sensitivity analysis further confirms OpsSkill maintains the top ranking under all 5 configurations (§5.10), proving robustness to hyperparameter choices. This robustness fundamentally stems from OpsSkill providing non-zero coverage on all evaluation dimensions—a *structural advantage* that no weight adjustment can offset.

E. Limitations

1) **Experiment scale**: 3-node cluster with 8 tasks across 3 fault domains (Pod, network, configuration level). While sufficient for validating core designs, this does not cover large-scale cluster (>100 nodes) cascading faults, resource contention, or diverse failure types (storage faults, security breaches, certificate expiration). Future work should validate OpsSkill's scalability on larger, more diverse fault sets.
2) **Scoring function**: Fixed weights in $\text{Score}(y, T)$ may not optimally match all task types—fault recovery emphasizes safety and recovery time, while configuration changes emphasize correctness and idempotency. Task-type-adaptive weight adjustment requires larger-scale experimental data for calibration.
3) **LLM dependency**: Skill compilation f_θ uses DeepSeek-V3; verification uses only heuristic rules without LLM verifiers. This limits verifier capability to predefined rule sets, unable to handle semantically complex verification (e.g., judging whether log anomaly patterns relate to the current fault). Future work may introduce LLMs into verification while preserving safety.
4) **Determinism**: Human-authored skill IRs and heuristic verifiers produce identical outputs for identical inputs (zero variance). While ensuring reproducibility, results do not reflect LLM compilation randomness. When f_θ generates skill IRs end-to-end, output variability may cause performance fluctuation, requiring further evaluation.

F. Threats to Validity

Internal. The scoring function may inherently favor structured-output methods—metrics like precondition ratio P_r and verification coverage V_{vis} are concepts defined by Typed Skill IR, on which non-structured methods (ReAct, Reflexion) necessarily score low. Mitigated by hidden verification as a unified, method-agnostic evaluation layer: all methods are evaluated via the same Kubernetes cluster state checks, ensuring final scores reflect actual operational results rather than formal structural matching. Moreover, the primary objective ($w_{primary} = 0.35$) and secondary objective ($w_{secondary} = 0.20$) comprise 55% of total score—these metrics are entirely cluster-state-based, independent of representation form.

External. Kubernetes-only validation; generalizability to other platforms (OpenStack, AWS, traditional servers) requires investigation. However, Typed Skill IR's design is platform-agnostic—its core abstractions (preconditions, action sequences, verifiers, rollback) apply to any scenario requiring safe automation. Adapting OpsSkill to new platforms mainly requires platform-specific $\kappa(E)$ and action executors; the compile–execute–optimize loop needs no modification. Cross-domain consistency experiments (Table 2, third row group) provide preliminary evidence of generalization across fault domains.

Construct. The 8 task cards are researcher-designed based on real operations experience, covering Pod management, network diagnosis, and configuration repair. However, these are not directly from production incident records, potentially limiting representativeness. Future work should incorporate public incident datasets [21] or industry-partnered real fault cases for more comprehensive evaluation.

VII. Conclusion

We presented OpsSkill, an autonomous operations skill learning framework for cloud-native systems. Targeting the core contradiction of existing LLM agents in high-risk operations—"strong capability but weak safety"—OpsSkill combines LLM knowledge compilation with formal safety guarantees through the "LLM Compile + Deterministic Execute" design philosophy. The three-layer core architecture—Typed Skill IR, risk-budget gated execution, and hidden multi-signal verification—respectively addresses structured representation of operations knowledge, safety constraints during execution, and reliability of result verification.

Main Experimental Findings. In 64 experiment trials (8 methods $\times$ 8 tasks) on a Kubernetes cluster (as a representative cloud-native platform):

- **Task completion quality**: OpsSkill achieves 90% composite score, exceeding all baselines by 50–65 pp (strongest baseline Reflexion achieves only 40%).
- **Safety**: Zero unsafe action rate (UAR = 0%) across all 24 executions, while ReAct's UAR reaches 5%.
- **Cross-domain generalization**: Consistent performance across Pod, network, and configuration fault domains (std $<$3%), validating Typed IR's domain invariance.
- **Optimizability**: Failure-driven optimization improves scores by 18% on average (15%$\rightarrow$33%) across 6 adversarial scenarios, validating Δ_σ's automatic mapping from failure signatures to structural edits.

Component Contribution Ranking. Ablation quantifies each component: Typed IR contributes most (-65%), showing structured knowledge representation is the fundamental performance driver; hidden verification follows (-30%), showing reliable result verification is key for closed-loop operations; policy gating has minimal score impact (-1%) but is critical for safety (UAR $+12\%$), embodying the design intent of trading minimal performance for critical safety guarantees.

Conclusion Robustness. Unified evaluation metrics (§5.8) confirm OpsSkill's advantage does not depend on specific scoring function design—performance ranking is unchanged when applying the same formula to all methods. Weight sensitivity (§5.10) further shows OpsSkill maintains the top ranking across all 5 configurations, with leads of 40%–55%. Propositions 1 and 2 provide semi-formal guarantees for gating safety and optimization convergence.

Future Directions.

1) **LLM compilation deepening**: Transitioning f_θ from human-authored to LLM end-to-end generation [22], researching how to maintain Typed IR quality under full automation.
2) **Large-scale validation**: Evaluating scalability on production-scale clusters ($>$100 nodes), replacing researcher-designed task cards with real incident data.
3) **Cross-platform generalization**: Adapting Typed Skill IR to OpenStack, AWS, and other cloud platforms to validate platform-agnostic design.
4) **Closed-loop self-optimization**: Implementing automatic detection of performance degradation and self-triggered optimization for continuous autonomous improvement.
5) **Human-in-the-loop integration**: Designing operator collaboration interfaces supporting manual review, intervention, and knowledge injection while maintaining automation efficiency.

References

[1] B. Burns, J. Beda, K. Hightower, and L. Evenson, *Kubernetes: Up and Running*, 3rd ed. O'Reilly Media, 2022.

[2] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.

[3] S. Ghosh, M. Shetty, C. Bansal, and S. Nath, "How to fight production incidents? An empirical study on a large-scale cloud service," in *Proceedings of the 13th Symposium on Cloud Computing (SoCC)*. ACM, 2022, pp. 126–141.

[4] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proceedings of the Eleventh International Conference on Learning Representations (ICLR)*, 2023.

[6] Q. Cheng, D. Sahoo, A. Saha, W. Yang, C. Liu, G. Woo, S. Saverese, and S. C. H. Hoi, "AI for IT operations (AIOps) on cloud platforms: Reviews, opportunities and challenges," in *arXiv preprint arXiv:2304.04661*, 2023.

[7] D. Roy, X. Zhang, R. Bhave, C. Bansal, P. Cardenas, S. Rajmohan, R. Nair, and V. Gopi, "Exploring LLM-based agents for root cause analysis," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE)*. ACM, 2024, pp. 208–219.

[8] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, "Identifying the risks of LM agents with an LM-emulated sandbox," *arXiv preprint arXiv:2309.15817*, 2023.

[9] J. Ye, S. Li, G. Li, C. Huang, S. Gao, Y. Wu, Q. Chen *et al.*, "ToolSword: Unveiling safety issues of large language models in tool learning across three stages," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. ACL, 2024, pp. 2147–2164.

[10] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[11] C. Bansal, S. Renganathan, A. Asudani, O. Midy, and M. Janakiraman, "DeCaf: Diagnosing and triaging performance issues in large-scale cloud services," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE)*. ACM, 2020, pp. 33–44.

[12] Y. Chen, H. Xie, M. Ma, Y. Kang, X. Gao, L. Shi, Y. Cao, X. Gao, H. Fan, M. Wen *et al.*, "Automatic root cause analysis via large language models for cloud incidents," in *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys)*. ACM, 2024, pp. 674–688.

[13] X. Zhou, G. Li, Z. Sun, Z. Liu, W. Chen, J. Wu, J. Liu, R. Feng, and G. Zeng, "D-Bot: Database diagnosis system using large language models," in *arXiv preprint arXiv:2312.01454*, 2023.

[14] M. Shetty, C. Bansal, S. P. Upadhyayula, R. Nair, and N. Nagappan, "AutoTSG: Learning and synthesis for incident troubleshooting," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 2022, pp. 290–302.

[15] K. An, F. Yang, J. Lu, L. Li, Z. Ren, H. Huang, L. Zhang, S. Rajmohan, Q. Zhang, V. Gopi *et al.*, "Nissist: An incident mitigation copilot based on troubleshooting guides," *arXiv preprint arXiv:2402.17531*, 2024.

[16] P. Las-Casas, A. G. Kumbhare, R. Fonseca *et al.*, "LLexus: An AI agent system for incident management," in *ACM SIGOPS Operating Systems Review*, vol. 58, no. 1. ACM, 2024, pp. 59–67.

[17] J. Mao, L. Li, Y. Gao, Z. Peng, S. He, C. Zhang *et al.*, "Agentic troubleshooting guide automation for incident management," in *arXiv preprint arXiv:2510.10074*, 2025.

[18] H. Xia, H. Wang, Z. Liu, Q. Yu, Y. Guo *et al.*, "SafeToolBench: Pioneering a prospective benchmark to evaluating tool utilization safety in LLMs," *Findings of EMNLP*, 2025.

[19] Chaos Mesh Authors, "Chaos Mesh: A cloud-native chaos engineering platform," https://chaos-mesh.org/, 2021, cNCF Incubating Project.

[20] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, "Chaos engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016.

[21] J. Chen, S. He, Y. Liu, L. Zhang, Y. Sui, and D. Zhang, "Continuous incident triage for large-scale online service systems," in *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2019, pp. 364–375.

[22] OpenAI, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.